Project Name
P228: Computer Vision Offside Monitoring for Queensland Rugby Union
Description
Manual testing of functionalities of the YOLO detection and classification features
Test Objective
Ensure that YOLO accurately detects and classifies rucks, lineouts, sports ball and players from pre-trained set of data

The following table contains manual testing conducted throughout the project. Additional technical automated testing was completed, and can be viewed in Submission\Code\src\unit_testing folder

Test Case ID	Feature	Pre Conditions	Test Steps	Expected Results	Post Conditions	Acceptance Criteria	Actual Results	Test Environment	Test Data	Test Approach
TC001	Verify that ruck model loads correctly	Ruck model is present in the models folder + ruck model is being called to load	1. Launch application, 2. Navigate to ruck detection, 3. Select and run sample video	No error messages are displayed and model starts detecting rucks	User is able to successfully test ruck videos using the ruck model	The model loads within reasonable time + model runs until user closes winow	Model loaded successfully	Vindows, Python	ruck.pt	Manual functional testing
TC002	Verify that lineout model loads correctly	Lineout model is present in the models folder + lineout model is being called to load	1. Launch application, 2. Navigate to lineout detection, 3. Select and run sample video	No error messages are displayed and model starts detecting lineouts	User is able to successfully test lineout videos using the lineout model	The model loads within reasonable time + model runs until user closes winow	Model loaded successfully	Windows + python	lineout.pt	Manual functional testing
TC003	Verify that ball model loads correctly	Ball model is present in the models folder + ball model is being called to load	1. Navigate to ball tracking, Select and run sample video	No error messages are displayed and model starts detecting ball	User is able to successfully test ball videos using the ball model	The model loads within reasonable time + model runs until user closes winow	Model loaded successfully	Windows + Python	ball.pt	Manual functional testing
TC004	Verify that YOLO ruck model actually detects rucks from video	Ruck video to be tested is in inference folder + ruck video path is called with ruck model	Load the ruck model, 2. Run inference on video	Window opens up with bounding box drawn around the ruck. No other bounding box should be present	User is able to visualize and track the ruck via bounding box as long as the ruck keeps happening	The bounding box detects rucks most of the time, with reasonable accuracy.	Bounding box draws around ruck with expected accuracy	Windows + YOLO, OpenCV	ruck video file	System testing, integration testing
TC005	Verify that YOLO lineout model actually detects lineouts from video	Lineout video to be tested is in inference folder + lineout video path is called with lineout model	1. Load the lineout video, 2. Run inference on video	Window opens up with bounding box drawn around the lineout. No other bounding box should be present	User is able to visualize and track the lineout via bounding box as long as the lineout keeps happening	The bounding box detects lineouts most of the time, with reasonable accuracy.	Bounding box draws around lineout with expected accuracy	Windows + YOLO, OpenCV	Lineout video file	System testing, integration testing
TC006	Verify that the video path file loads correctly	User provides file path that exists in the folder	1. Pass file path to script, 2. Load video or image	File (img or vid) opens in a new window	User is able to view img or vid and use this with ruck and lineout models	A separate file displaying the full img or vid opens within reasonable time	Image or video loads successfully	Windows, OpenCV installed	Video and image files	Functional Testing
TC007	Verify that the colour detection file runs without errors	Colour detection script and models are presented	1. Load the colour detection file, 2. Run inference with test video	No error messages and correct colour detection overlays are shown	User is able to see colour overlays indicating different features	Colour overlays appear within reasonable time and match expected features	Colour detection works without crashing and overlays correctly	Windows , OpenCV	Colour detection video	Functional + system testing

TC008	Verify that colour detection is accurate due to dominant colour and classification functions working correctly	Team colours of the two playing teams are specified. Number of k clusters and brightness are specified.	1. Run the colour detection script, 2. Input the video with known team jersey colours, 3. Observe classification results	Program detects dominant colour of each players jersey and classifies into 1 of 2 teams. Bounding box around the player is the same colour as dominant colour of players jersey	User is able to clearly see which team each player belongs to.	Players are correctly classified into team based on jersey colour.	Players correctly detected and classified by jersey colour with expected accuracy	Windows, Python and OpenCV	Team 1: black [0, 0, 0]. Team 2: white [255, 255, 255]	Functional Testing
TC009	Verify that referees and other non-player objects are omitted from detection	Team colours of the two playing teams are specified	1. Run detection script, 2. Observe whether referees and orhter non-players are detected or omitted	Every detected player is classified either into team 1 or 2. No other person is detected.	There is no bounding box around non-players and user can clearly see what team each player belongs to	100% of non-players are omitted from detection.	No bounding box drawn on referees or other non-payers	Windows + python + detection and classification model	Team 1: black [0, 0, 0]. Team 2: white [255, 255, 255]	Functional Testing
TC010	Test model on long video without staging images	Model is trained and operational, long video input is available	1. Load a long video with no staging images, 2. Run model	Model processes without crashing	Model finishes without halting	Model continues to run through video without any errors occurring	Originally crashed with a NaN in intersection point	Python, video processing environment with model loaded	Long rugby match video without stages	Manual Exploratory Test
TC011	Image display functionality	Model is running, image display logic is implemented	1. Add image display to code, 2. Run Model	Frames are displayed properly	Images show per frame during processing	No Freezing, Image output updates correctly	Initially non-responsive, fixed by ensuring computation before display	Python, OpenCV	Rugby match frames	Manual debug and test
TC012	Ruck Detection Model	Rick model loaded and running	1. Run ruck detection, 2. Check accuracy of box and feet placement	Ruck boxes appear at correct time and location	Accurate bounding boxes with each frame	Boxes match visual ground truth during rucks	Ruck calculated from last image, fixed by moving box logic	Python + Yolo model	Rugby clips with visible rucks	Manual testing with frame-by-frame validation
TC013	Ruck detection + ball release interaction	Both ruck and ball release logic implemented	1. Trigger ball release code, 2. Observe if ruck box is detected during same frames	Ruck box and ball release detection can occur simultaneously	Synchronised detection	No conflict between ruck and ball release logic	No ruck box during ball release, fixed by fallback logic	Python	Video clips with both ball release and ruck events	Integrated system test
TC014	Ball release timing	Ball detection logic active	1. Play frames where ball is released, 2. Evaluate if release time is detected correctly	Ball release frame is detected correctly	Release time recorded	Consistent and correct frame marking of ball release	Initially poor detection, proved by using tracking logic of the ball	Python, Object tracking libraries	Rugby clips with ball release actions	Manual with tracking validation
TC015	Colour-based team detection	Region of interest is defined	1. Detect colour in large ROI, Identify which team player belongs to	Player's team colour is identified correctly	Accurate team classification	ROI correctly identifies jersey colours	Large box mainly included and detected grass noise, fixed by implementing smaller ROI	Python with OpenCV	Frames with multiple players	Visual ROI testing and validation

TC016	Ball object detection	Model trained to detect ball	1. Play video with visible ball, 2. Check if ball is correctly identified	Ball is consistently and accuately detected	Detected ball locations per frame	False positive rate is low, true positive rate is high	Many false detections, improved with more annotations and training of model	Yolo detection in Python	Frame dataset with annotated balls	Iterative training and testing
TC017	Model adaptability across camera angles	Multiple camera angles available	1. Run model on videos from different angles, 2. Track object detection accuracy	Model detects objects regardless of camera angle	Robust detection across perspectives	Consistent detection on all camera types	Miss-detections occurred, fixed by expanding dataset and retraining model	Python + Roboflow for training	Multi-angle rugby videos	Cross-angle robustness testing
TC018	Line detection using Hough transform	Hough line algorithm implemented for offside or field lines	1. Rub model on angled video footage, 2. Observe line detection behaviour	Lines are detected accurately dispite camera angle	Detected lines available for further processing	Detected lines correctly align with actual field markings	Failed on angled lines; camera angle curved line, Hough failed	Python with OpenCV (HoughLine Transform)	Angled field view frames	Visual analysis, switched to contour detection
TC019	Performance of rugby.pt model (multi class detection)	Combined yolo model (rugby.pt) trained for all classes	1. Run rugby.pt on full field footage, 2. Evaluate ruck and lineout detection	Model detects rucks and lineouts anywhere on the field	All relevant events are detected	No blind spots on field edges	Poor detection on most classes in the model; improved by separating the model into individual classes	Yolo model in python (combined vs. separate approach	Rugby footage with action across full field	Model restructuring and comparitive performance test
TC020	Offside player detection	Offside zone calculated; player detections running	1. Run offside detection on player positions, check flagged players	Only players from the correct team are flagged when offside	Players identified and marked appropriately	False positives minimized through team identification	Initially flagged all players in the area; fixed with team based colour filtering	Python, OpenCV, team colour classification logic	Rugby frames with players in offside zones	Functional Testing